Lab 2

$ setup + Makefiles + recursion

Platform and WSL notes

macOS: Apple Containers requires macOS `15.0` or later.

WSL failure reported in office hours: `wsl --update` restored a broken WSL installation.

If an update does not help: save needed files, then rebuild the Linux distribution.

```
wsl --list --verbose
wsl --unregister <distro>
wsl --update
wsl --list --online
wsl --install Ubuntu
```

`wsl --unregister` deletes that distribution and its Linux files.

(repeat) Forgot your sudo password?

WSL: close WSL terminals, open PowerShell, then run:

```
wsl -u root
passwd <your-username>
```

Exit the root shell and reopen WSL.

Apple Containers on macOS: run from the host:

```
container exec -u root <container-name> passwd <your-username>
```

VS Code AI features

Open **Settings**, search `chat.disableAIFeatures`, and enable it.

VS Code hides built-in chat and inline suggestions, then disables Copilot extensions.

Reasons:

- **It is annoying**
- **you let AI complete your assignments -> you might fail the exams**

Makefiles

Labs include a small `Makefile`.

Point: stop typing long, annoying `gcc` commands. Type `make` instead.

A Makefile can automate any command, including compilation, testing, and cleanup.

Live demo during lab.

Makefile comments

You do not need to memorize the next slides. Use them to get the gist of how the lab Makefiles work.

`#` starts a comment. `make` ignores the line.

```
1   # This is a comment line
2
3   # define variables so it is easier to make changes
4   CC=gcc
5   CFLAGS=-g -Wall -std=c99 -fsanitize=address,undefined
6   TARGETS=foo bar
7
8   all: $(TARGETS)
9
10  # foo: foo.c
11  #   $(CC) $(CFLAGS) -o foo foo.c
12
13  # bar: bar.c
14  #   $(CC) $(CFLAGS) -o $@ $@.c
15
16  $(TARGETS): %: %.c
17    $(CC) $(CFLAGS) -o $@ $@.c
18
19  clean:
20    $(RM) -rf *.o *~ $(TARGETS) a.out
```

Makefile variables

Variables keep compiler options and program names in one place.

`CC` selects the compiler. `CFLAGS` holds compiler flags. `TARGETS` lists the programs.

```
1   # This is a comment line
2
3   # define variables so it is easier to make changes
4   CC=gcc
5   CFLAGS=-g -Wall -std=c99 -fsanitize=address,undefined
6   TARGETS=foo bar
7
8   all: $(TARGETS)
9
10  # foo: foo.c
11  #   $(CC) $(CFLAGS) -o foo foo.c
12
13  # bar: bar.c
14  #   $(CC) $(CFLAGS) -o $@ $@.c
15
16  $(TARGETS): %: %.c
17    $(CC) $(CFLAGS) -o $@ $@.c
18
19  clean:
20    $(RM) -rf *.o *~ $(TARGETS) a.out
```

The default target

`all` is the first build target. It depends on every name in `$(TARGETS)`.

Running `make` builds `foo` and `bar`.

```
1   # This is a comment line
2
3   # define variables so it is easier to make changes
4   CC=gcc
5   CFLAGS=-g -Wall -std=c99 -fsanitize=address,undefined
6   TARGETS=foo bar
7
8   all: $(TARGETS)
9
10  # foo: foo.c
11  #   $(CC) $(CFLAGS) -o foo foo.c
12
13  # bar: bar.c
14  #   $(CC) $(CFLAGS) -o $@ $@.c
15
16  $(TARGETS): %: %.c
17    $(CC) $(CFLAGS) -o $@ $@.c
18
19  clean:
20    $(RM) -rf *.o *~ $(TARGETS) a.out
```

Explicit target examples

These commented lines show one recipe per program.

Remove # to use explicit recipes. The pattern rule below avoids repeating them.

```
# This is a comment line

# define variables so it is easier to make changes
CC=gcc
CFLAGS=-g -Wall -std=c99 -fsanitize=address,undefined
TARGETS=foo bar

all: $(TARGETS)

# foo: foo.c
#   $(CC) $(CFLAGS) -o foo foo.c

# bar: bar.c
#   $(CC) $(CFLAGS) -o $@ $@.c

$(TARGETS): %: %.c
  $(CC) $(CFLAGS) -o $@ $@.c

clean:
  $(RM) -rf *.o *~ $(TARGETS) a.out
```

Pattern rule and recipe

The pattern rule matches every program in `TARGETS` with a matching `.c` file.

`%` means the program name. `$@` means the target currently built. The recipe line starts with a tab.

```
1   # This is a comment line
2
3   # define variables so it is easier to make changes
4   CC=gcc
5   CFLAGS=-g -Wall -std=c99 -fsanitize=address,undefined
6   TARGETS=foo bar
7
8   all: $(TARGETS)
9
10  # foo: foo.c
11  #   $(CC) $(CFLAGS) -o foo foo.c
12
13  # bar: bar.c
14  #   $(CC) $(CFLAGS) -o $@ $@.c
15
16  $(TARGETS): %: %.c
17    $(CC) $(CFLAGS) -o $@ $@.c
18
19  clean:
20    $(RM) -rf *.o *~ $(TARGETS) a.out
```

The clean target

`clean` removes build output and editor backup files.

Run `make clean` before a fully fresh build. Check the command before changing its file patterns.

```
# This is a comment line

# define variables so it is easier to make changes
CC=gcc
CFLAGS=-g -Wall -std=c99 -fsanitize=address,undefined
TARGETS=foo bar

all: $(TARGETS)

# foo: foo.c
#   $(CC) $(CFLAGS) -o foo foo.c

# bar: bar.c
#   $(CC) $(CFLAGS) -o $@ $@.c

$(TARGETS): %: %.c
  $(CC) $(CFLAGS) -o $@ $@.c

clean:
  $(RM) -rf *.o *~ $(TARGETS) a.out
```

Debugging

You can use these to help on the recursion in the Lab 2 problems:

VS Code debugger:

- Open **Run and Debug** in the sidebar
- Set a breakpoint near the recursive call
- Press `F5`, then inspect **Call Stack** while stepping
- Each call has its own copy of variables you can inspect

ASan:

- Compile with `-fsanitize=address` flag. You can add to Makefile
- This will show you what line your program segfaulted on.

Browser option: `from0.dev/tracer` visualizes function calls and returns.

Questions?

Office hours (Hudson)

Monday, 12:15 PM to 2:15 PM @ ITE 214

Discord

Ask any time

I will try to be quick!

Credits:

Slides made with Typst (typst.app) w/ help from OpenAI Codex